# Automi e Reti Logiche

## Raccolta Completa: 4 Lezioni ed Esercizi

Documento consolidato con testo integrale, tabelle, formule e tutti gli schemi visivi.
Formattato appositamente per l'ingestione e lo studio su Google NotebookLM.

✨ Fonte Unica per Google NotebookLM

# Automa di Moore e Riconoscitore di Sequenze (101)

> **Obiettivo del Progetto**
>
> Progettare un automa a stati finiti deterministico che riceve in ingresso uno stream continuo di bit binari (0 e 1) e genera in uscita una segnalazione di **Riconoscimento (R)** ogni volta che rileva la sequenza target `101`, emettendo **Non Riconosciuto (NR)** in tutti gli altri istanti.

---

## 1. Il Modello di Moore vs il Modello di Mealy

La differenza cardine risiede nel punto in cui viene calcolata l'uscita:

```
flowchart TD
    subgraph MEALY [Automa di Mealy]
        S_Mealy[Stato Interno S] --> Calc_U_Mealy[Funzione g]
        In_Mealy[Ingresso I] --> Calc_U_Mealy
        Calc_U_Mealy --> Out_Mealy["Uscita U(t) = g(S, I)"]
    end

    subgraph MOORE [Automa di Moore]
        S_Moore[Stato Interno S] --> Calc_U_Moore[Funzione g]
        Calc_U_Moore --> Out_Moore["Uscita U(t) = g(S)"]
    end
```

| Caratteristica | Automa di Mealy | Automa di Moore |
| --- | --- | --- |
| **Formula Uscita** | $U(t) = g(S(t), I(t))$ | $U(t) = g(S(t))$ |
| **Collocazione Uscita** | Sugli **archi** di transizione ( `i / u` ) | All'interno dei **nodi/stati** ( `Stato / U` ) |
| **Numero di Stati** | Spesso minore ($N$ stati) | Spesso maggiore ($N + 1$ stati) per isolare l'uscita |
| **Risposta Temporale** | Immediata all'arrivo dell'ingresso | Ritardata di un ciclo di clock (stabile e senza glitch) |

---

## 2. Riconoscimento della Sequenza `101` : Approccio di Mealy

Nel modello di Mealy, l'uscita di riconoscimento R viene emessa direttamente sull'arco di transizione nel momento stesso in cui arriva l'ultimo bit `'1'` .

### Definizione degli Stati:

- **s1 (Reset / Vuoto):** Nessun bit valido ricevuto. Attesa del primo `'1'` .
- **s2 (Visto `1` ):** Primo bit valido agganciato. Attesa dello `'0'` .
- **s3 (Visto `10` ):** Primi due bit validi agganciati. Attesa del terzo bit `'1'` .

```
↔stateDiagram-v2
    direction LR
    [*] --> S1
    S1 --> S1 : 0 / NR
    S1 --> S2 : 1 / NR
    S2 --> S2 : 1 / NR
    S2 --> S3 : 0 / NR
    S3 --> S1 : 0 / NR
    S3 --> S2 : 1 / R (Sequenza 101 agganciata!)
```

---

## 3. Riconoscimento della Sequenza `101` : Approccio di Moore

Nell'automa di Moore, l'uscita **non può trovarsi sull'arco**. Per segnalare il riconoscimento R, l'automa deve necessariamente entrare in uno **stato dedicato** la cui etichetta interna produce R.

### Definizione degli Stati (Prefissi Minimi):

- **S0 (Prefisso ϵ / Vuoto)** → **Uscita: NR:** Nessun carattere utile della sequenza presente.
- **S1 (Prefisso `1` )** → **Uscita: NR:** Ricevuto `'1'`, in attesa di `'0'`.
- **S2 (Prefisso `10` )** → **Uscita: NR:** Ricevuto `'10'`, in attesa di `'1'`.
- **S3 (Prefisso `101` - Riconosciuto!)** → **Uscita: R:** Sequenza `101` completata con successo!

```
stateDiagram-v2
    direction LR

    [*] --> S0

    state S0 {
        description: "Uscita = NR"
    }
    state S1 {
        description: "Uscita = NR"
    }
    state S2 {
        description: "Uscita = NR"
    }
    state S3 {
        description: "Uscita = R (OK!)"
    }

    S0 --> S0 : 0
    S0 --> S1 : 1

    S1 --> S1 : 1
    S1 --> S2 : 0

    S2 --> S0 : 0
    S2 --> S3 : 1

    S3 --> S2 : 0 (il bit finale '1' + '0' forma '10')
    S3 --> S1 : 1 (il bit finale è '1')
```

## Logica delle Transizioni di Ritorno da S3:

Quando l'automa si trova nello stato di successo S3 (ha appena visto `101`):

1. **Se riceve `0`:** La sequenza diventa `...1 0 1 0`. Gli ultimi due bit sono `10`, che rappresentano esattamente il prefisso dello stato S2! L'automa salta quindi direttamente a **S2** senza perdere il progresso.
2. **Se riceve `1`:** La sequenza diventa `...1 0 1 1`. L'ultimo bit è `1`, quindi l'automa transita nello stato **S1**.

## 4. Tabella di Transizione e Trasformazione (Moore)

| Stato Attuale | Uscita Emessa | Prossimo Stato (Ingresso = 0) | Prossimo Stato (Ingresso = 1) |
|---|---|---|---|
| **S0** (Vuoto) | **NR** | S0 | S1 |
| **S1** (Visto `1`) | **NR** | S2 | S1 |
| **S2** (Visto `10`) | **NR** | S0 | S3 |
| **S3** (Visto `101`) | **R** | S2 | S1 |

---

## 5. Esempio Pratico con Traccia Temporale

Supponiamo di inviare la stringa binaria di ingresso: `1 0 1 0 1 1 0`

| Passo Temporale t | t0 | t1 | t2 | t3 | t4 | t5 | t6 | t7 |
|---|---|---|---|---|---|---|---|---|
| **Bit in Ingresso** | - | **1** | **0** | **1** | **0** | **1** | **1** | **0** |
| **Stato Raggiunto** | S0 | S1 | S2 | **S3** | S2 | **S3** | S1 | S2 |
| **Uscita Moore** | NR | NR | NR | **R** | NR | **R** | NR | NR |

> **Osservazione Chiave**

> Notiamo che la sequenza `1 0 1 0 1` contiene **due riconoscimenti** a t3 e a t5 perché il bit `'1'` centrale a t3 funge contemporaneamente sia da chiusura del primo `101` sia da inizio del secondo `101` (**sovrapposizione / overlapping**).

---

# Automi a Stati Finiti e Diagrammi di Stato

> **Cos'è un Automa a Stati Finiti (FSM)?**
>
> Un **Automa a Stati Finiti** (*Finite State Machine - FSM*) è un modello matematico di computazione costituito da un insieme discreto e finito di **stati**, da un alfabeto di **ingressi**, da un alfabeto di **uscite** e da regole che governano le **transizioni** da uno stato all'altro in risposta agli stimoli esterni.

---

## 1. Definizione Formale dell'Automa

Formalmente, un automa a stati finiti deterministico è rappresentato dalla sestupla:

$M = \langle I, U, S, f, g, s0 \rangle$

- **I (Alfabeto degli Ingressi):** Insieme finito dei possibili simboli/stimoli in ingresso.
- **U (Alfabeto delle Uscite):** Insieme finito delle possibili risposte o azioni generate.
- **S (Insieme degli Stati):** Insieme finito degli stati interni che il sistema può assumere.
- **f (Funzione di Transizione):** Regola che associa allo stato corrente e all'ingresso il prossimo stato: $f : S \times I \rightarrow S$
- **g (Funzione di Trasformazione/Uscita):**
  - Per automi di **Mealy**: $g : S \times I \rightarrow U$ (l'uscita è funzione di stato e ingresso).
  - Per automi di **Moore**: $g : S \rightarrow U$ (l'uscita dipende solo dallo stato).
- **s0 ∈ S (Stato Iniziale):** Lo stato in cui si trova il sistema all'avvio.

---

## 2. Anatomia del Diagramma degli Stati (Grafo delle Transizioni)

Il **diagramma degli stati** è una rappresentazione visuale a grafo orientato:

```
stateDiagram-v2
    direction LR
    [*] --> S1 : Avvio (Stato Iniziale)
    S1 --> S1 : i_invariante / u_invariante (Cappio)
    S1 --> S2 : i_evento / u_risposta (Arco orientato)
    S2 --> [*] : Conclusione (Stato Finale)
```

| Elemento Grafico | Significato Logico | Notazione |
|---|---|---|
| **Cerchio (Nodo / Bolla)** | Rappresenta un particolare stato interno si. | `(s1)` |
| **Freccia senza origine** | Indica univocamente lo **Stato Iniziale** (s0 o s1). | ▶ `(s1)` |
| **Doppio Cerchio** | Indica uno **Stato Finale / Terminale** (nei riconoscitori o processi a termine). | `((s_end))` |
| **Arco Orientato** | Indica una transizione valida da uno stato sorgente a uno stato destinazione. | → |
| **Cappio (Self-loop)** | Transizione che riporta nello stesso stato se l'ingresso non produce avanzamento. | ↻ |
| **Etichetta sull'Arco** | Specifica la condizione di attivazione e l'effetto prodotto: `Ingresso / Uscita` (sintassi Mealy). | `1 EUR / Eroga` |

---

## 3. Caso di Studio: Distributore Automatico con Resto

Analizziamo il sistema reale modellato negli appunti: un **distributore automatico di bibite** (costo bevanda: **2 EUR**) che accetta monete da **1 EUR** e **2 EUR** ed è in grado di erogare la bibita scelta (Aranciata, Cola) ed erogare il corretto resto.

## 1. Definizione delle Variabili di Sistema

- **Vettore Ingressi I = {i1, i2}:**
  - i1 (Moneta inserita): Vi1 = {1 EUR, 2 EUR}
  - i2 (Selezione bevanda): Vi2 = {Sel. Aranciata, Sel. Cola, Nessuna}
- **Vettore Uscite U = {u1, u2}:**
  - u1 (Erogazione bevanda): Vu1 = {Lattina Aranciata, Lattina Cola, Null}
  - u2 (Erogazione resto): Vu2 = {Resto 0 EUR, Resto 1 EUR, Resto 2 EUR}
- **Insieme degli Stati S = {s1, s2}:**
  - **s1 (Credito 0 EUR - Attesa prima moneta):** Stato iniziale di riposo.
  - **s2 (Credito 1 EUR - Attesa seconda moneta):** Il sistema memorizza che è già stato inserito 1 EUR.

---

## 4. Diagramma degli Stati del Distributore (Modello di Mealy)

```
stateDiagram-v2
    direction LR

    [*] --> S1: Accensione

    state S1 {
        description: "Credito: 0 EUR"
    }
    state S2 {
        description: "Credito: 1 EUR"
    }

    S1 --> S2: Inserito 1€ / (Bibita: NULL, Resto: 0€)
    S1 --> S1: Inserito 2€ + Sel.Cola / (Bibita: Lattina Cola, Resto: 0€)
    S1 --> S1: Inserito 2€ + Sel.Aranciata / (Bibita: Lattina Aranciata, Resto: 0€)

    S2 --> S1: Inserito 1€ + Sel.Cola / (Bibita: Lattina Cola, Resto: 0€)
    S2 --> S1: Inserito 1€ + Sel.Aranciata / (Bibita: Lattina Aranciata, Resto: 0€)
    S2 --> S1: Inserito 2€ + Sel.Cola / (Bibita: Lattina Cola, Resto: 1€)
    S2 --> S1: Inserito 2€ + Sel.Aranciata / (Bibita: Lattina Aranciata, Resto: 1€)
```

## Spiegazione dei Percorsi e Risoluzione del Resto:

1. **Partenza da s1 (Credito 0 EUR):**
   - Se l'utente inserisce **2 EUR** e preme Cola: il costo (2 EUR) è coperto. Il sistema eroga subito la Cola con **Resto 0 EUR** e **ritorna in s1**, pronto per un nuovo cliente.
   - Se l'utente inserisce **1 EUR**: il credito è insufficiente. Il sistema transita nello stato **s2** (*credito accumulato = 1 EUR*) e l'uscita è `(Bibita: NULL, Resto: 0 EUR)`.
2. **Nello stato s2 (Credito 1 EUR accumulato):**
   - Se l'utente inserisce un altro **1 EUR** (totale 2 EUR) e seleziona Cola: viene erogata la **Lattina Cola** con **Resto 0 EUR**, e il sistema ritorna allo stato di riposo **s1**.
   - Se l'utente inserisce una moneta da **2 EUR** (totale accumulato 1 + 2 = 3 EUR) e seleziona Cola: il sistema eroga la **Lattina Cola** e contemporaneamente restituisce **Resto 1 EUR** (3 – 2 = 1 EUR), ritornando in **s1**.

> **Principio di Ciclicità**
>
> Nei sistemi reattivi continui (come un distributore o un controllore di semaforo), l'automa non termina in uno "stato finale chiuso" ma deve **sempre resettarsi allo stato iniziale** dopo aver portato a termine l'erogazione.

---

# Riconoscitori di Sequenze con e senza Sovrapposizione (*Overlapping*)

> **Definizione del Problema**
>
> Dato uno stream sequenziale di simboli binari in ingresso, un **riconoscitore di sequenza** ha il compito di analizzare i bit in tempo reale ed emettere un segnale di validità (**OK**) nell'istante esatto in cui viene completata una precisa sottostringa target (ad esempio `010`).
>
> La distinzione fondamentale riguarda come viene trattato l'ultimo bit della sequenza riconosciuta: può essere riutilizzato per avviare una nuova sequenza (**con sovrapposizione / concatenato**) oppure deve essere scartato/azzerato (**senza sovrapposizione / non concatenato**).

---

## 1. Il Concetto di Concatenazione (Overlapping)

Consideriamo la sequenza target **`010`** e inviamo in ingresso la stringa di test:

Input: 0 1 0 1 0

```
flowchart TD
    subgraph OVERLAP [Modalità CON Sovrapposizione / Overlapping]
        direction TB
        S1["[ 0  1  0 ] 1  0  --> 1° Riconoscimento (bit 1-3)"]
        S2["0  1 [ 0  1  0 ] --> 2° Riconoscimento (bit 3-5)"]
        RES1["Totale Riconoscimenti: 2"]
    end

    subgraph NO_OVERLAP [Modalità SENZA Sovrapposizione / Non-Overlapping]
        direction TB
        N1["[ 0  1  0 ] consumato!"]
        N2["Restano: 1  0 (non formano 010)"]
        RES2["Totale Riconoscimenti: 1"]
    end
```

- **Senza Sovrapposizione (Non-Overlapping):** Una volta riconosciuta la sottostringa, tutti i bit che ne facevano parte vengono considerati "consumati". L'automa resetta la ricerca ripartendo completamente da zero.
- **Con Sovrapposizione (Overlapping):** I caratteri finali della sequenza appena riconosciuta possono sovrapporsi ai caratteri iniziali della sequenza successiva, permettendo rilevamenti multipli ravvicinati.

---

## 2. Progettazione dell'Automa per la Sequenza `010`

### Definizione Formale:

- **Alfabeto Ingressi:** $I = \{0, 1\}$
- **Alfabeto Uscite:** $U = \{OK, NR\}$ (OK = Riconosciuto, NR = Non Riconosciuto)
- **Stati del Sistema (S):**
  - **s1 (Attesa 1° bit):** Nessun bit valido. Si aspetta `'0'`.
  - **s2 (Visto `0` - Attesa 2° bit):** Primo bit valido agganciato. Si aspetta `'1'`.
  - **s3 (Visto `01` - Attesa 3° bit):** Primi due bit validi agganciati. Si aspetta `'0'`.

---

## 3. Automa SENZA Sovrapposizione (Non-Overlapping)

Se la sequenza viene completata, l'automa emette OK e torna allo stato iniziale s1:

```
↔stateDiagram-v2
    direction LR

    [*] --> s1: Reset
    s1 --> s1: 1 / NR
    s1 --> s2: 0 / NR

    s2 --> s2: 0 / NR (lo '0' mantiene l'attesa di '1')
    s2 --> s3: 1 / NR

    s3 --> s1: 1 / NR (sequenza rotta: visto '011')
    s3 --> s1: 0 / OK (Sequenza '010' completata! Reset a s1)
```

> **Comportamento della Transizione da s3**
>
> Quando arriva lo `'0'` in s3, la sequenza `010` è completa: l'uscita è **OK**, ma l'arco punta a **s1**, azzerando la memoria.

---

## 4. Automa CON Sovrapposizione (Overlapping)

Se la sequenza viene completata, l'ultimo bit `'0'` è già il primo bit valido della sequenza successiva: l'automa transita quindi direttamente a s2!

```
stateDiagram-v2
    direction LR

    [*] --> s1: Reset
    s1 --> s1: 1 / NR
    s1 --> s2: 0 / NR

    s2 --> s2: 0 / NR
    s2 --> s3: 1 / NR

    s3 --> s1: 1 / NR
    s3 --> s2: 0 / OK (Sequenza '010' completata! Lo '0' riparte da s2)
```

> **La Differenza Cruciale**
>
> L'unica differenza topologica tra i due grafi è la destinazione dell'arco `0 / OK` uscente da s3:
>
> - Senza sovrapposizione: punta a **s1**.
> - Con sovrapposizione: punta a **s2**.

---

## 5. Tabella Comparativa di Traccia Temporale

Inviamo la sequenza di bit: `0 1 0 1 0 0 1 0`

| Istante t | t1 | t2 | t3 | t4 | t5 | t6 | t7 | t8 |
|---|---|---|---|---|---|---|---|---|
| **Bit in Ingresso** | **0** | **1** | **0** | **1** | **0** | **0** | **1** | **0** |
| **Stato (Senza Overlap)** | s2 | s3 | s1 | s1 | s2 | s2 | s3 | s1 |
| **Uscita (Senza Overlap)** | NR | NR | **OK** | NR | NR | NR | NR | **OK** |
| **Stato (Con Overlap)** | s2 | s3 | s2 | s3 | s2 | s2 | s3 | s2 |
| **Uscita (Con Overlap)** | NR | NR | **OK** | NR | **OK** | NR | NR | **OK** |

### Risultato del Test:

- **Senza Overlap:** 2 riconoscimenti (all'istante t3 e all'istante t8).
- **Con Overlap: 3 riconoscimenti** (agli istanti t3, t5 e t8), perché a t5 viene catturata la sequenza sovrapposta `0 1 0` a cavallo dei bit 3, 4 e 5!

---

## 6. Ambiti Applicativi Reali

- **Protocolli di Rete (Framing):** Nei protocolli come **HDLC** (High-Level Data Link Control) o **PPP**, i pacchetti sono delimitati dalla sequenza speciale di flag `01111110`. L'automa del ricevitore deve individuare la fine di un frame e l'inizio del successivo senza perdere bit.
- **Compilatori e Tokenizer:** Nell'analisi lessicale di codice sorgente, le parole chiave vengono separate senza sovrapposizione ( `if`, `while`, identificatori).
- **Crittografia e Analisi di Flussi:** Ricerca di firme malevole o pattern all'interno di pacchetti ispezionati da firewall/IDS (Intrusion Detection System).

# Tabelle di Transizione e Trasformazione negli Automi

> **Ruolo delle Tabelle nella Modellazione**
>
> Mentre il **diagramma degli stati (grafo)** offre una visione intuitiva e visiva del funzionamento del sistema, le **tabelle di transizione e trasformazione** costituiscono la rappresentazione algebrico-matriciale rigorosa e completa. Sono il passaggio fondamentale indispensabile per procedere alla progettazione e alla sintesi circuitale hardware (con flip-flop e porte logiche) o all'implementazione software (con matrici o istruzioni `switch-case`).

---

## 1. Struttura Matriciale delle Tabelle

Un sistema a stati finiti descrive la propria evoluzione attraverso due matrici bidimensionali:

```
flowchart LR
    subgraph TAB_TRANS [Tabella di Transizione di Stato]
        direction TB
        T1["Righe: Stato Attuale S(t)"]
        T2["Colonne: Ingressi I(t)"]
        T3["Celle: STATO FUTURO S(t+1) = f(S, I)"]
        T1 --- T2 --- T3
    end

    subgraph TAB_TRAS [Tabella di Trasformazione delle Uscite]
        direction TB
        U1["Righe: Stato Attuale S(t)"]
        U2["Colonne: Ingressi I(t)"]
        U3["Celle: USCITA EMESSA U(t) = g(S, I)"]
        U1 --- U2 --- U3
    end
```

---

## 2. Caso di Studio: Distributore di Bibite (Costo 2 EUR)

Analizziamo il sistema reale tratto dagli appunti: un distributore automatico in cui una lattina costa **2 EUR**.

### 1. Variabili di Ingresso (I)

- $i1 \in \{1\ EUR, 2\ EUR\}$ (Valore della moneta introdotta)
- $i2 \in \{ON, OFF\}$ (Pulsante di erogazione premuto o non premuto)

Combinazioni di ingresso simultanee: $(i1, i2) \in \{(1\ Euro, ON), (2\ Euro, ON), (1\ Euro, OFF), (2\ Euro, OFF)\}$

### 2. Variabili di Uscita (U)

- $u1 \in \{ON, OFF\}$ (Attivazione del motorino di erogazione della lattina)

### 3. Insieme degli Stati (S)

- **s1 (Attesa prima moneta):** Credito attuale = 0 EUR.
- **s2 (Attesa seconda moneta):** Credito attuale = 1 EUR (manca 1 EUR al totale).
- **s3 (Attesa click pulsante):** Credito attuale = 2 EUR raggiunto (credito sufficiente).
- **s4 (Lattina in uscita):** Fase di rilascio prodotto e reset del sistema.

---

## 3. Tabella di Transizione (Stato Futuro $S(t + 1)$)

Specifica in quale stato transita il sistema al ciclo successivo in base allo stato presente e all'azione dell'utente:

| Stato Corrente S(t) | Ingresso: (1 EUR, ON) | Ingresso: (2 EUR, ON) | Ingresso: (1 EUR, OFF) | Ingresso: (2 EUR, OFF) |
|---|---|---|---|---|
| **s1** (Credito 0 EUR) | s2 (Attesa 2ª moneta) | s4 (Lattina in uscita) | s2 (Attesa 2ª moneta) | s3 (Attesa click) |
| **s2** (Credito 1 EUR) | s4 (Lattina in uscita) | s4 (Lattina in uscita) | s3 (Attesa click) | s3 (Attesa click) |
| **s3** (Credito 2 EUR) | s4 (Lattina in uscita) | s4 (Lattina in uscita) | s3 (Attesa click) | s3 (Attesa click) |
| **s4** (Erogazione) | s1 (Reset a 0 EUR) | s1 (Reset a 0 EUR) | s1 (Reset a 0 EUR) | s1 (Reset a 0 EUR) |

---

## 4. Tabella di Trasformazione (Uscita U(t))

Specifica lo stato dell'uscita (erogazione lattina: ON oppure OFF):

| Stato Corrente S(t) | Ingresso: (1 EUR, ON) | Ingresso: (2 EUR, ON) | Ingresso: (1 EUR, OFF) | Ingresso: (2 EUR, OFF) |
|---|---|---|---|---|
| **s1** (Credito 0 EUR) | OFF | ON | OFF | OFF |
| **s2** (Credito 1 EUR) | ON | ON | OFF | OFF |
| **s3** (Credito 2 EUR) | ON | ON | OFF | OFF |
| **s4** (Erogazione) | ON | ON | OFF | OFF |

---

## 5. Corrispondenza con il Grafo degli Stati

```
stateDiagram-v2
    direction TB

    [*] --> s1: Reset iniziale

    s1 --> s2: (1€, OFF) o (1€, ON) / OFF
    s1 --> s3: (2€, OFF) / OFF
    s1 --> s4: (2€, ON) / ON

    s2 --> s3: (1€, OFF) / OFF
    s2 --> s4: (1€, ON) o (2€, ON) / ON

    s3 --> s4: (qualsiasi, ON) / ON
    s3 --> s3: (qualsiasi, OFF) / OFF

    s4 --> s1: Fine erogazione (Reset automatico)
```

> **Come leggere la matrice**
>
> Se il sistema si trova in **s2** (l'utente ha già inserito 1 EUR) e introduce un altro **1 EUR** premendo contemporaneamente il pulsante di erogazione (**ON**):
>
> - Si incrocia la riga s2 con la colonna (1 EUR, ON).
> - Nella Tabella di Transizione troviamo: **s4** (il sistema passa allo stato di erogazione).
> - Nella Tabella di Trasformazione troviamo: **ON** (il motorino della lattina viene attivato immediatamente).

---

## 6. Dalla Tabella alla Sintesi Circuitale (Flip-Flop)

Per trasformare queste tabelle in un circuito elettronico reale:

1. **Codifica degli Stati in Binario:** Avendo 4 stati (s1, s2, s3, s4), servono: K = ⌈log2 4⌉ = 2 bit di memoria (Flip-Flop Q1, Q0)
    - s1 = 00
    - s2 = 01
    - s3 = 10
    - s4 = 11
2. **Mappe di Karnaugh:** Si compilano le mappe di Karnaugh per ricavare le funzioni minime booleane degli ingressi dei flip-flop (es. tipo D o tipo JK) e dell'uscita u1.
3. **Realizzazione:** Si collegano i flip-flop e la rete combinatoria di porte logiche (AND, OR, NOT) ottenuta dalla semplificazione.

---